STATIC CODE ANALYSIS

LAB 5 — Python Code Quality, Security and Style

Name	Kaushik Balaji	**SRN**	PES2UG24CS220
Section	5D	**Program**	expense_tracker.py

Objective

To improve the quality, security and maintainability of a Python expense-tracking program by using Pylint, Bandit and Flake8 to identify issues, applying fixes, and rerunning the tools to verify the updated source.

Program and execution

The assigned file was **expense_tracker.py**. It records expenses, totals amounts by category, updates a monthly budget, calculates the remaining budget, identifies the largest expense, checks a configured sync password, and provides a helper for creating a report file. The program was run from PowerShell inside a Python virtual environment. The demonstration output showed Food = 750, Travel = 1200 and Bills = 999; after setting the budget to 5000, the remaining budget was 2051. The negative expense amount was rejected.

Tools used

Tool	Purpose in this lab
Pylint	Checks code quality, naming, documentation, and common design or programming practices.
Bandit	Scans Python source for common security risks and unsafe coding patterns.
Flake8	Checks Python style and formatting, including whitespace and line-length rules.

Analysis and fixes

The first scan reported naming and documentation warnings in Pylint, formatting violations in Flake8, and security findings in Bandit. The fixes below address the original findings recorded during the run. Original line numbers are approximate to the initial source/report and can differ after edits.

Issue / finding	Tool / original location	Fix applied	Verification
Hardcoded demo password	Bandit B105; line 5	Read from the BANK_SYNC_PASSWORD environment variable instead of embedding it in source.	Final Bandit scan: no issues identified.
Hardcoded Linux temporary directory	Bandit B108; line 8	Use tempfile.gettempdir() via os.path.join() for a platform-appropriate temporary directory.	Final Bandit scan: no issues identified.
Unsafe tempfile.mktemp()	Bandit B306; line 55	Use NamedTemporaryFile() to create the temporary file securely; retain it with delete=False when needed.	Final Bandit scan: no issues identified.
Shell command constructed with os.system()	Bandit B605; line 61 (high)	Removed the unused file-opening helper from the demo, avoiding shell command construction.	Final Bandit scan: no issues identified.
Overly broad exception handling	Pylint W0718; initial line 41	Check whether the expense list is empty before calling max(); return None for the empty-list case.	Empty-list behavior checked in program output.
Inconsistent naming and missing docstrings	Pylint C0103/C0116; multiple lines	Use snake_case for functions, add docstrings, and store the budget in a dictionary to avoid global.	Final Pylint score: 10.00/10.
Whitespace, blank-line, comment-spacing and long-line violations	Flake8 E203/E302/E261/E501	Correct spacing, add a blank line, fix inline-comment spacing, and split the long print statement.	Final flake8 command produced no output.

Verification summary: The latest captured results show Pylint = 10.00/10, Bandit = no issues identified, and Flake8 = no output from the direct command. The program's demonstration output was also checked. Static analysis does not prove the program is free of every defect.

Reflection and conclusion

Note about the handout: The supplied four-page student handout requires reflection answers in its Deliverables section, but it does not include an explicit numbered list of reflection questions. To avoid inventing questions and presenting them as verbatim handout content, the following responses address the learning outcomes and reflection requirement stated in the handout.

1. What did static analysis contribute to the quality and maintainability of the program?

Static analysis found issues without needing to execute every code path. Pylint highlighted inconsistent naming, missing documentation and overly broad exception handling. Flake8 identified formatting problems. Bandit highlighted patterns that could expose the program to security risks. Fixing these findings made the source clearer, more consistent and easier to maintain.

2. How do Pylint, Bandit and Flake8 differ, and why were all three useful?

Pylint focuses on general code quality and programming practices; Flake8 checks style and PEP 8-related formatting; Bandit focuses on common security-sensitive patterns. Their roles overlap only partly, so one tool's clean result does not replace the others. Running all three provided complementary checks.

3. How were issues prioritized and how were the fixes verified?

Security-related findings were prioritized first, especially the original shell-command construction, unsafe temporary-file creation, hardcoded password and temporary-directory path. Code-quality and style findings were then corrected. The program was run again, and all three tools were rerun. The final captured results were Pylint 10.00/10, Bandit with no issues identified, and no output from Flake8.

4. What limitations or lessons remain after this lab?

Static analysis can report potential risks that require context, and a clean report does not guarantee the absence of runtime, logic or design defects. Program execution and appropriate tests are still necessary. Secrets should be configured outside source code, and security warnings should be investigated rather than silenced without understanding them.

Conclusion

The expense tracker was refactored to address more than four original findings. The final evidence shows a Pylint score of 10.00/10, no Bandit issues, and no Flake8 output. These results indicate that the reported quality, security-pattern and formatting findings were addressed in the final version; they should be considered alongside functional testing.

Final verification evidence

The screenshots below are terminal captures from the user's execution. They are included as evidence and kept in their original color. They show the program output, the final Pylint/Bandit result, and the final Flake8 command returning without reported violations.

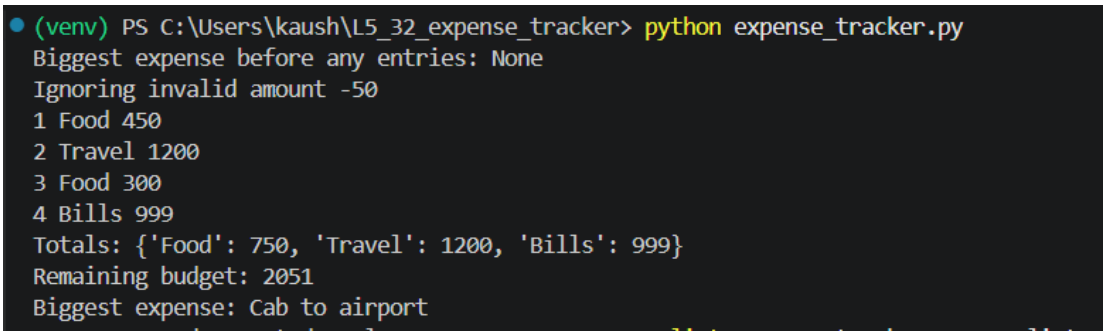


Figure 1. Program execution showing expense entries, category totals, remaining budget and largest expense.

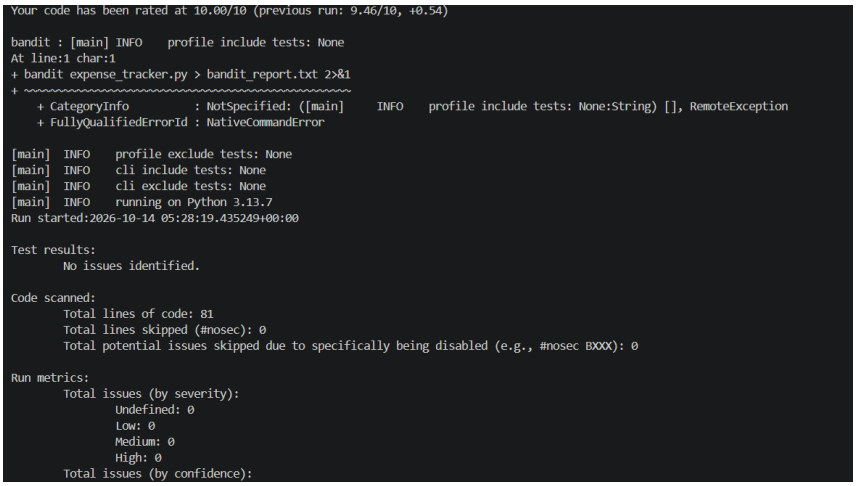


Figure 2. Final Pylint score (10.00/10) and Bandit result (no issues identified).

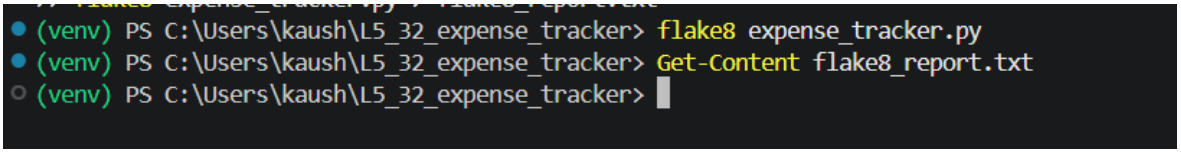


Figure 3. Final Flake8 command and empty report output.